

10.回顾Java对象的比较

【本节目标】

1. Java中对象的比较
2. 集合框架中PriorityQueue的比较方式
3. 模拟实现PriorityQueue

1. PriorityQueue中插入对象

上节课我们讲了优先级队列，优先级队列在插入元素时有个要求：插入的元素不能是null或者元素之间必须要能够进行比较，为了简单起见，我们只是插入了Integer类型，那优先级队列中能否插入自定义类型对象呢？

```
1 class Card {
2     public int rank; // 数值
3     public String suit; // 花色
4
5     public Card(int rank, String suit) {
6         this.rank = rank;
7         this.suit = suit;
8     }
9 }
10
11 public class TestPriorityQueue {
12     public static void TestPriorityQueue()
13     {
14         PriorityQueue<Card> p = new PriorityQueue<>();
15         p.offer(new Card(1, "♠"));
16         p.offer(new Card(2, "♠"));
17     }
18
19     public static void main(String[] args) {
20         TestPriorityQueue();
21     }
22 }
```

优先级队列底层使用堆，而向堆中插入元素时，为了满足堆的性质，必须要进行元素的比较，而此时Card是没有办法直接进行比较的，因此抛出异常。

2. 元素的比较

2.1 基本类型的比较

在Java中，基本类型的对象可以直接比较大小。

```
1 public class TestCompare {
2     public static void main(String[] args) {
3         int a = 10;
4         int b = 20;
5         System.out.println(a > b);
6         System.out.println(a < b);
7         System.out.println(a == b);
8
9         char c1 = 'A';
10        char c2 = 'B';
11        System.out.println(c1 > c2);
12        System.out.println(c1 < c2);
13        System.out.println(c1 == c2);
14
15        boolean b1 = true;
16        boolean b2 = false;
17        System.out.println(b1 == b2);
18        System.out.println(b1 != b2);
19    }
20 }
```

2.2 对象比较的问题

```
1 class Card {
2     public int rank;        // 数值
3     public String suit;     // 花色
4
5     public Card(int rank, String suit) {
6         this.rank = rank;
7         this.suit = suit;
8     }
9 }
10
11 public class TestPriorityQueue {
12     public static void main(String[] args) {
13         Card c1 = new Card(1, "♠");
14         Card c2 = new Card(2, "♠");
```

```

15         Card c3 = c1;
16
17         //System.out.println(c1 > c2);    // 编译报错
18         System.out.println(c1 == c2);    // 编译成功 ----> 打印false, 因为c1和c2指向
        的是不同对象
19         //System.out.println(c1 < c2);    // 编译报错
20         System.out.println(c1 == c3);    // 编译成功 ----> 打印true, 因为c1和c3指向的
        是同一个对象
21     }
22 }

```

c1、c2和c3分别是Card类型的引用变量，上述代码在比较编译时：

c1 > c2 编译失败

c1 == c2 编译成功

c1 < c2 编译失败

从编译结果可以看出，Java中引用类型的变量不能直接按照 > 或者 < 方式进行比较。那为什么==可以比较？

因为：对于用户实现自定义类型，都默认继承自Object类，而Object类中提供了equal方法，而==默认情况下调用的就是equal方法，但是该方法的比较规则是：没有比较引用变量引用对象的内容，而是直接比较引用变量的地址，但有些情况下该种比较就不符合题意。

```

1 // Object中equal的实现，可以看到：直接比较的是两个引用变量的地址
2 public boolean equals(Object obj) {
3     return (this == obj);
4 }

```

3. 对象的比较

有些情况下，需要比较的是对象中的内容，比如：向优先级队列中插入某个对象时，需要对按照对象中内容来调整堆，那该如何处理呢？

3.1 覆写基类的equals

```

1 public class Card {
2     public int rank;    // 数值
3     public String suit;    // 花色
4
5     public Card(int rank, String suit) {
6         this.rank = rank;
7         this.suit = suit;
8     }
9 }

```

```

8      }
9
10     @Override
11     public boolean equals(Object o) {
12         // 自己和自己比较
13         if (this == o) {
14             return true;
15         }
16
17         // o如果是null对象, 或者o不是Card的子类
18         if (o == null || !(o instanceof Card)) {
19             return false;
20         }
21
22         // 注意基本类型可以直接比较, 但引用类型最好调用其equal方法
23         Card c = (Card)o;
24         return rank == c.rank
25             && suit.equals(c.suit);
26     }
27 }

```

注意：一般覆写 equals 的套路就是上面演示的

1. 如果指向同一个对象，返回 true
2. 如果传入的为 null，返回 false
3. 如果传入的对象类型不是 Card，返回 false
4. 按照类的实现目标完成比较，例如这里只要花色和数值一样，就认为是相同的牌
5. 注意下调用其他引用类型的比较也需要 equals，例如这里的 suit 的比较
6. 一般情况下 我们借助编译器自动生成

覆写基类equal的方式虽然可以比较，但缺陷是：**equal只能按照相等进行比较，不能按照大于、小于的方式进行比较。**

3.2 基于Comparable接口类的比较

Comparable是JDK提供的泛型的比较接口类，源码实现具体如下：

```

1 public interface Comparable<E> {
2     // 返回值:
3     // < 0: 表示 this 指向的对象小于 o 指向的对象
4     // == 0: 表示 this 指向的对象等于 o 指向的对象
5     // > 0: 表示 this 指向的对象大于 o 指向的对象
6     int compareTo(E o);
7 }

```

对用户自定义类型，如果要想按照大小与方式进行比较时：在定义类时，实现Comparable接口即可，然后在类中重写compareTo方法。

```
1 public class Card implements Comparable<Card> {
2     public int rank;           // 数值
3     public String suit;        // 花色
4
5     public Card(int rank, String suit) {
6         this.rank = rank;
7         this.suit = suit;
8     }
9
10    // 根据数值比较，不管花色
11    // 这里我们认为 null 是最小的
12    @Override
13    public int compareTo(Card o) {
14        if (o == null) {
15            return 1;
16        }
17        return rank - o.rank;
18    }
19
20    public static void main(String[] args){
21        Card p = new Card(1, "♠");
22        Card q = new Card(2, "♠");
23        Card o = new Card(1, "♠");
24        System.out.println(p.compareTo(o));           // == 0, 表示牌相等
25        System.out.println(p.compareTo(q));           // < 0, 表示 p 比较小
26        System.out.println(q.compareTo(p));           // > 0, 表示 q 比较大
27    }
28 }
```

Comparable是java.lang中的接口类，可以直接使用。

3.3 基于比较器比较

按照比较器方式进行比较，具体步骤如下：

- 用户自定义比较器类，实现Comparator接口

```
1 public interface Comparator<T> {
2     // 返回值:
3     // < 0: 表示 o1 指向的对象小于 o2 指向的对象
```

```
4 // == 0: 表示 o1 指向的对象等于 o2 指向的对象
5 // > 0: 表示 o1 指向的对象大于 o2 指向的对象
6 int compare(T o1, T o2);
7 }
```

- 注意：区分Comparable和Comparator。
- 覆写Comparator中的compare方法

```
1 import java.util.Comparator;
2
3 class Card {
4     public int rank;        // 数值
5     public String suit;     // 花色
6
7     public Card(int rank, String suit) {
8         this.rank = rank;
9         this.suit = suit;
10    }
11 }
12
13 class CardComparator implements Comparator<Card> {
14     // 根据数值比较，不管花色
15     // 这里我们认为 null 是最小的
16     @Override
17     public int compare(Card o1, Card o2) {
18         if (o1 == o2) {
19             return 0;
20         }
21
22         if (o1 == null) {
23             return -1;
24         }
25
26         if (o2 == null) {
27             return 1;
28         }
29
30         return o1.rank - o2.rank;
31     }
32
33     public static void main(String[] args){
34         Card p = new Card(1, "♠");
35         Card q = new Card(2, "♠");
36         Card o = new Card(1, "♠");
37     }
```

```

38      // 定义比较器对象
39      CardComparator cmptor = new CardComparator();
40
41      // 使用比较器对象进行比较
42      System.out.println(cmptor.compare(p, o));           // == 0, 表示牌
相等
43      System.out.println(cmptor.compare(p, q));           // < 0, 表示 p 比较小
44      System.out.println(cmptor.compare(q, p));           // > 0, 表示 q 比较大
45  }
46  }

```

注意：Comparator是java.util 包中的泛型接口类，使用时必须导入对应的包。

3.4 三种方式对比

覆写的方法	说明
Object.equals	因为所有类都是继承自 Object 的，所以直接覆写即可，不过只能比
Comparable.compareTo	需要手动实现接口，侵入性比较强，但一旦实现，每次用该类都有顺 内部顺序
Comparator.compare	需要实现一个比较器对象，对待比较类的侵入性弱，但对算法代码实 强

4. 观察源码当中PriorityQueue的比较方式

观察源码

5. 使用PriorityQueue创建大小堆，解决TOPK问题

```

1  //使用比较器创建小根堆
2  class LessIntComp implements Comparator<Integer>{
3      @Override
4      public int compare(Integer o1, Integer o2) {
5          return o1 - o2;
6      }
7  }
8  //使用比较器创建大根堆
9  class GreaterIntComp implements Comparator<Integer>{
10     @Override
11     public int compare(Integer o1, Integer o2) {
12         return o2 - o1;
13     }
14 }
15

```

```

16 public class TestDemo<E> {
17
18     //求最小的K个数，通过比较器创建大根堆
19     public static int[] smallestK(int[] array, int k) {
20         if(k <= 0) {
21             return new int[k];
22         }
23         GreaterIntComp greaterCmp = new GreaterIntComp();
24         PriorityQueue<Integer> maxHeap = new PriorityQueue<>(greaterCmp);
25         //先将前K个元素，创建大根堆
26         for(int i = 0; i < k; i++) {
27             maxHeap.offer(array[i]);
28         }
29         //从第K+1个元素开始，每次和堆顶元素比较
30         for (int i = k; i < array.length; i++) {
31             int top = maxHeap.peek();
32             if(array[i] < top) {
33                 maxHeap.poll();
34                 maxHeap.offer(array[i]);
35             }
36         }
37         //取出前K个
38         int[] ret = new int[k];
39         for (int i = 0; i < k; i++) {
40             int val = maxHeap.poll();
41             ret[i] = val;
42         }
43         return ret;
44     }
45
46     public static void main(String[] args) {
47         int[] array = {4,1,9,2,8,0,7,3,6,5};
48         int[] ret = smallestK(array,3);
49         System.out.println(Arrays.toString(ret));
50     }
51 }

```

完